Modeling and Simulation — Lesson 4

Simple Rules, Complex Worlds: Cellular Automata

Václav B. Petrák

Faculty of Biomedical Engineering
Czech Technical University

March 19, 2026

Conway's Game of Life

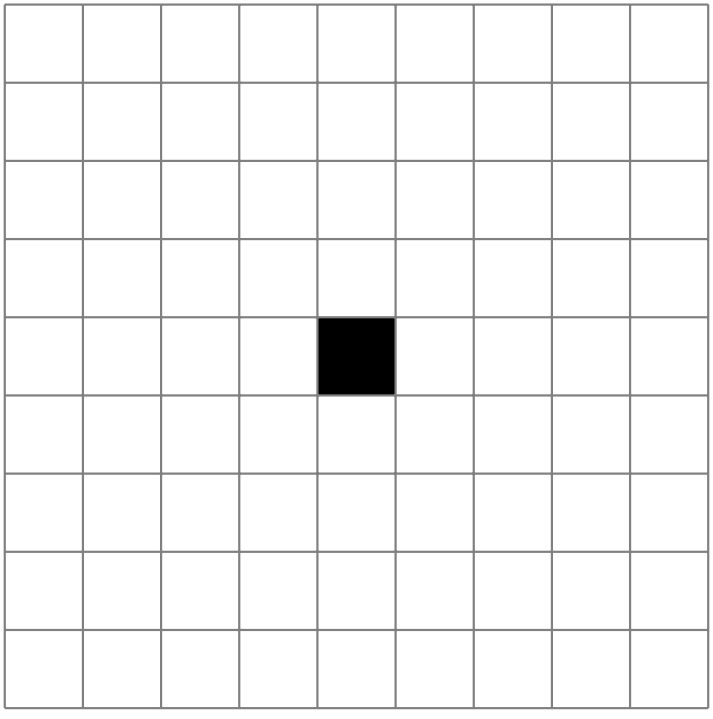
Conway's Game of Life

Try to google: "Conway's Game of Life"

Overview of Conway's Game of Life

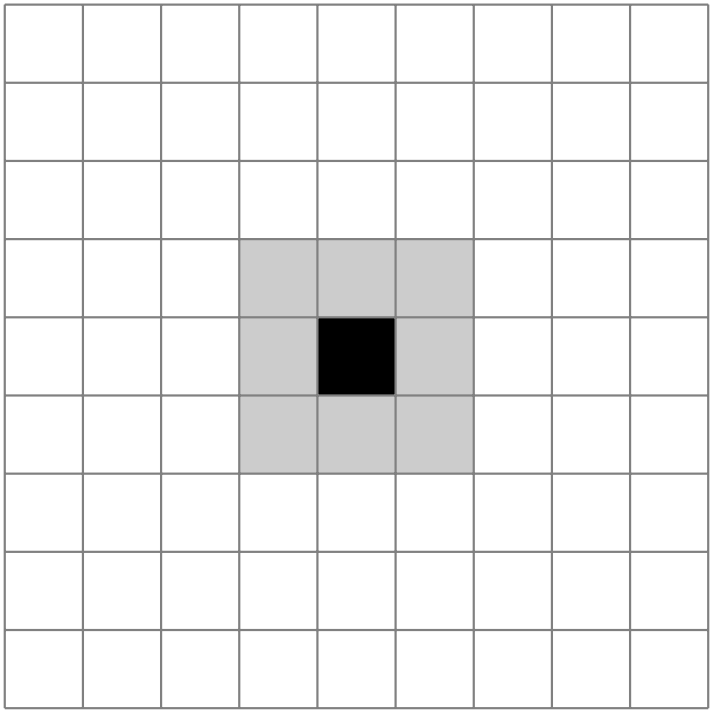
- **Overview:**
  - Created by the mathematician John Horton Conway in 1970.
  - A zero-player game where evolution is determined by its initial state, without stochastic elements.
  - The game unfolds on an infinite grid.
  - Each cell can be either alive or dead.
- **Rules of Evolution:**
  1. Any live cell with fewer than two live neighbors dies.
  2. Any live cell with two or three live neighbors lives on to the next generation.
  3. Any live cell with more than three live neighbors dies.
  4. Any dead cell with exactly three live neighbors becomes a live cell.

Each cell can be either alive or dead.



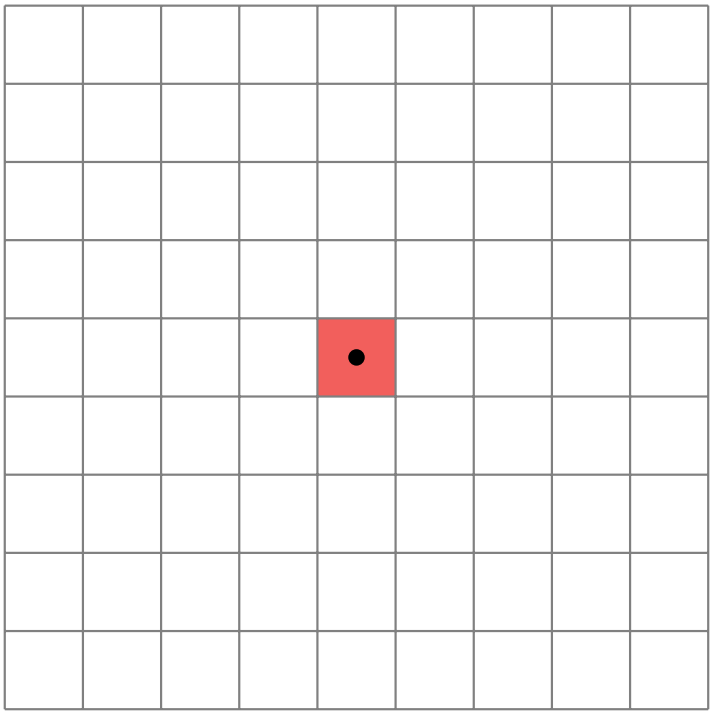
The status of each cell changes each generation.

Status depends on the status of the cell and its 8 neighbors.



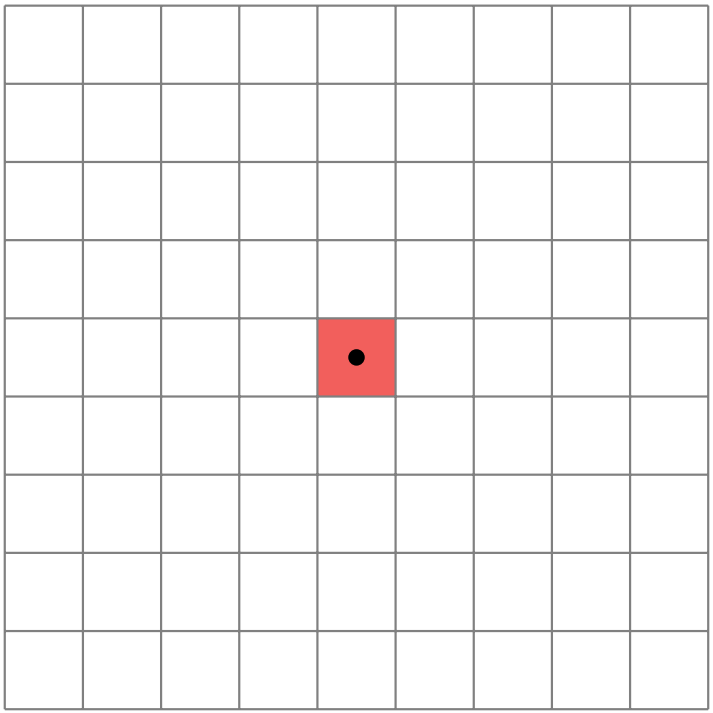
At each step, transitions occur according to four rules

Rule 1: Any live cell with fewer than two live neighbors dies, as if by underpopulation.



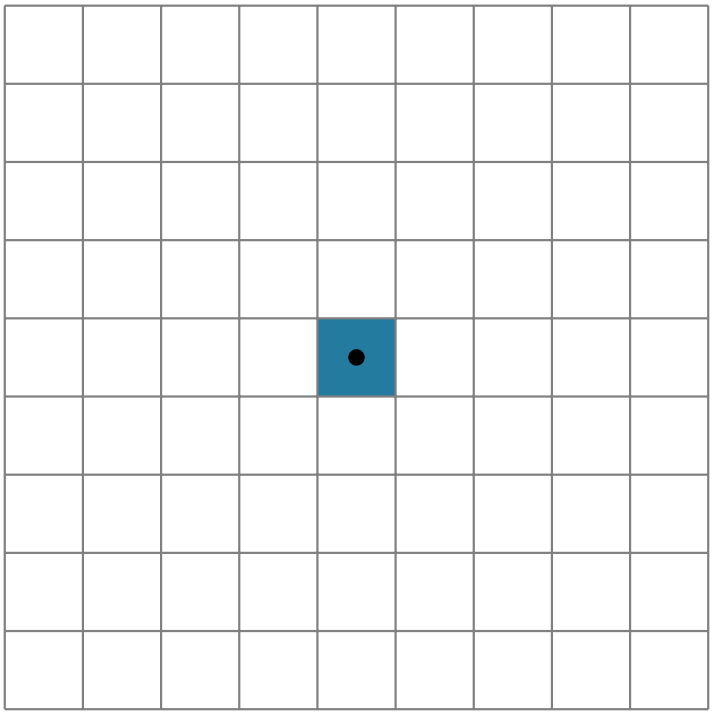
At each step, transitions occur according to four rules

Rule 1: Any live cell with fewer than two live neighbors dies, as if by underpopulation.



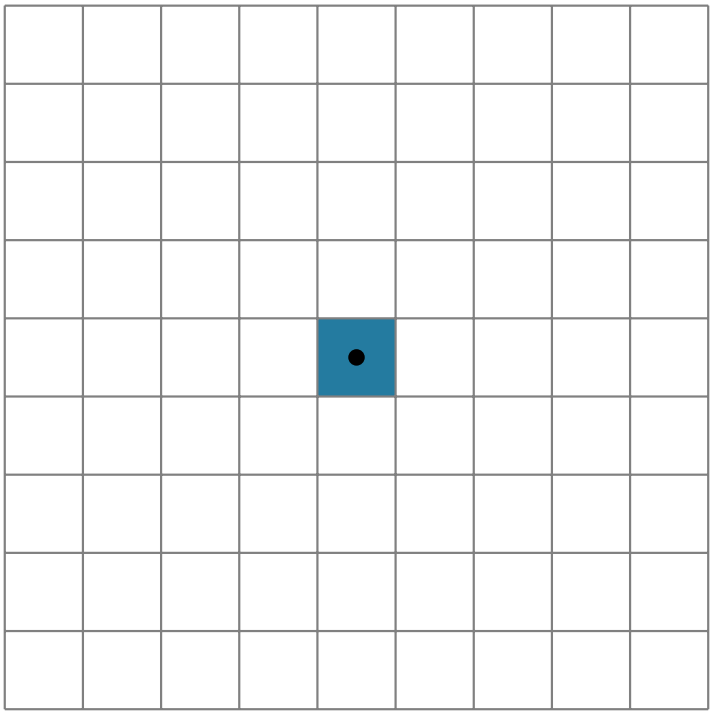
At each step, transitions occur according to four rules

Rule 2: Any live cell with two or three live neighbors lives on to the next generation.



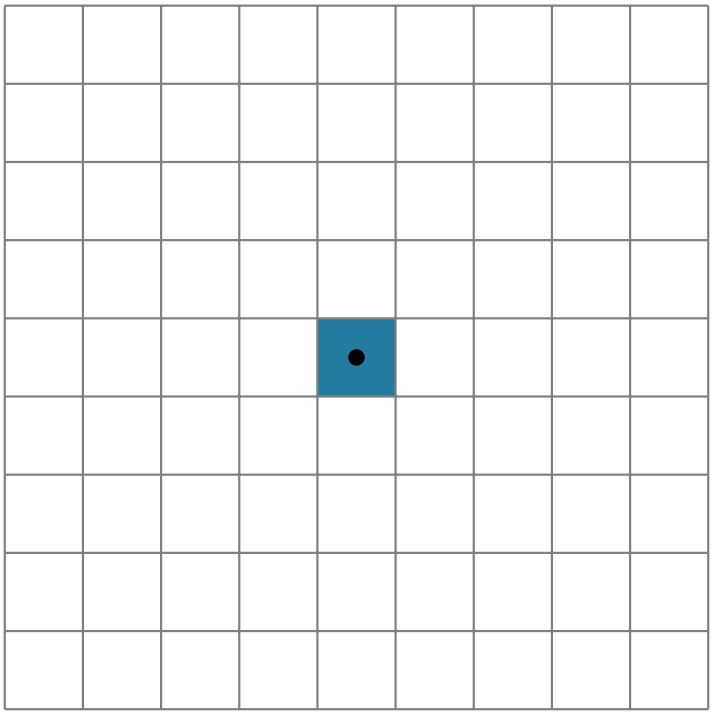
At each step, transitions occur according to four rules

Rule 2: Any live cell with two or three live neighbors lives on to the next generation.



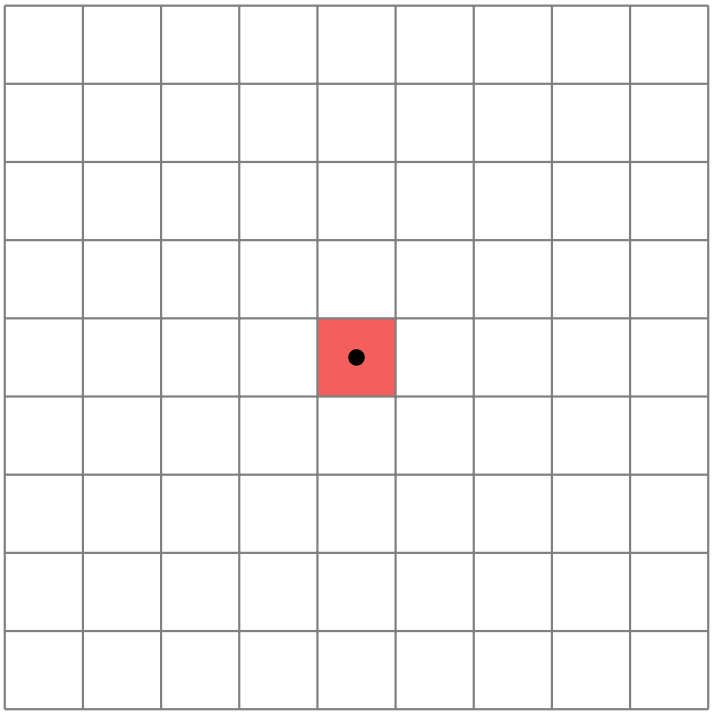
At each step, transitions occur according to four rules

Rule 2: Any live cell with two or three live neighbors lives on to the next generation.



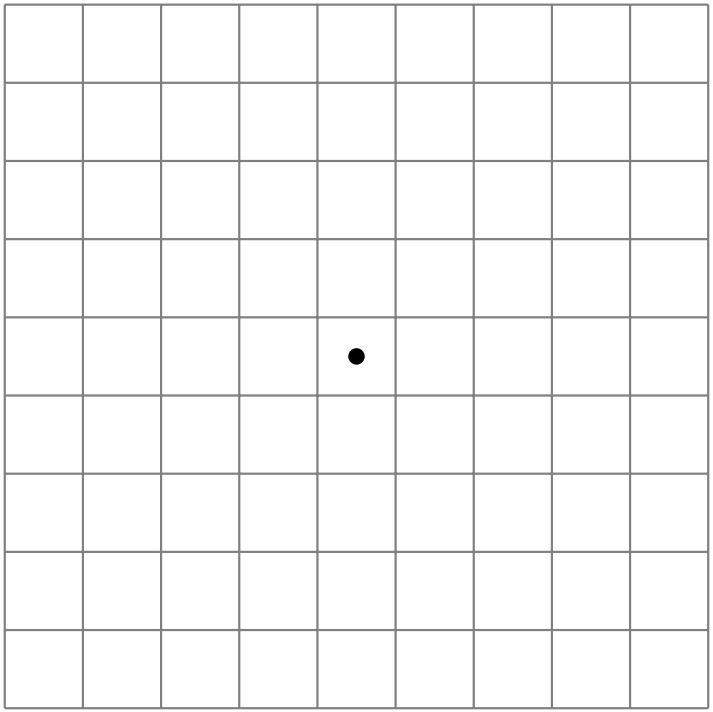
At each step, transitions occur according to four rules

Rule 3: Any live cell with more than three live neighbors dies, as if by overpopulation.



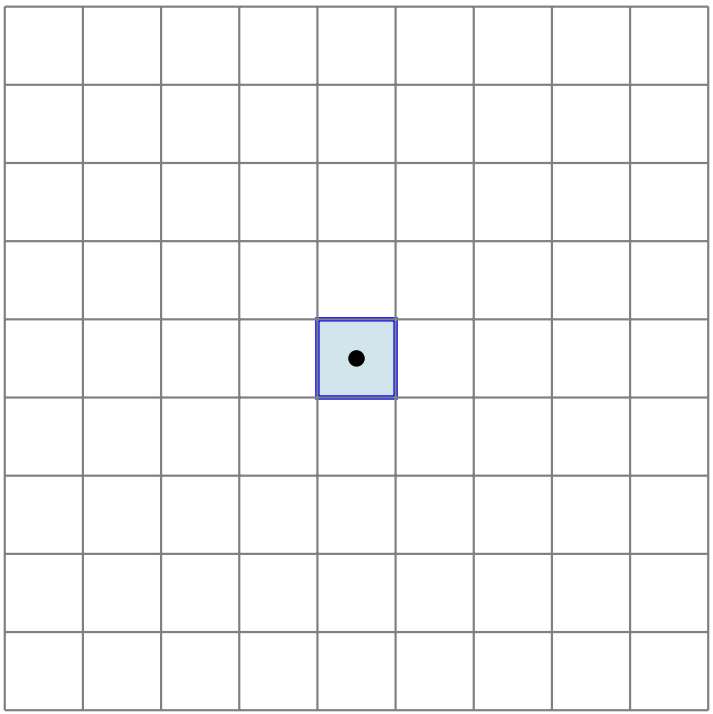
At each step, transitions occur according to four rules

Rule 4: Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.



At each step, transitions occur according to four rules

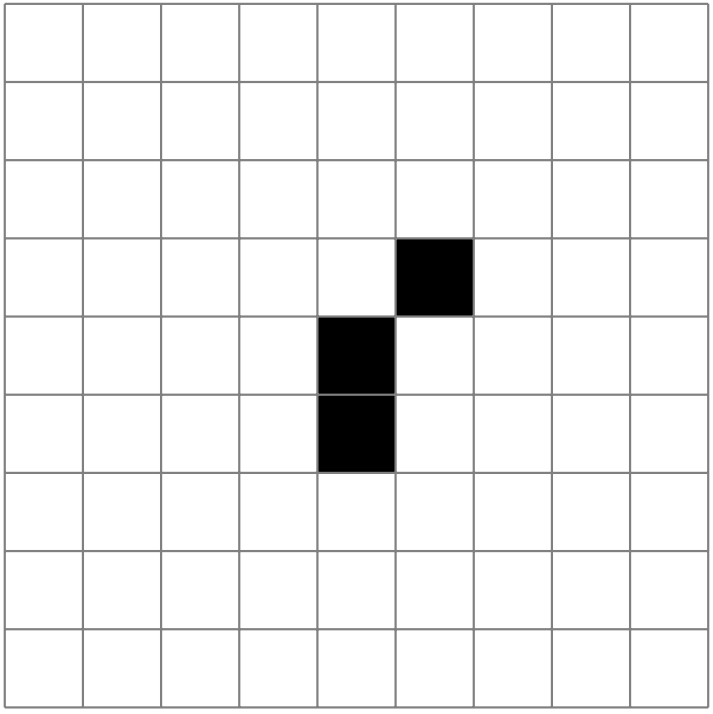
Rule 4: Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.



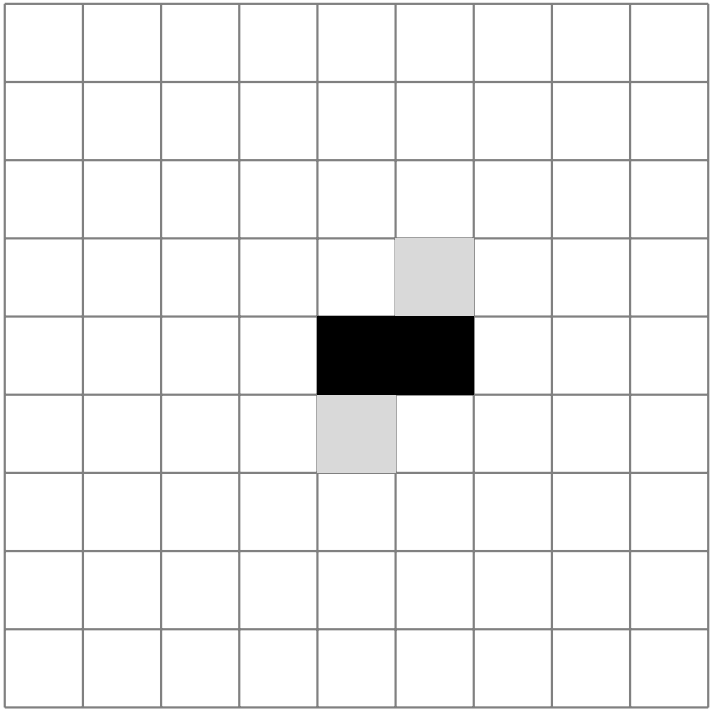
Frame Title

Starting from the initial configuration, these rules are applied, and the game board evolves, playing the game by itself.

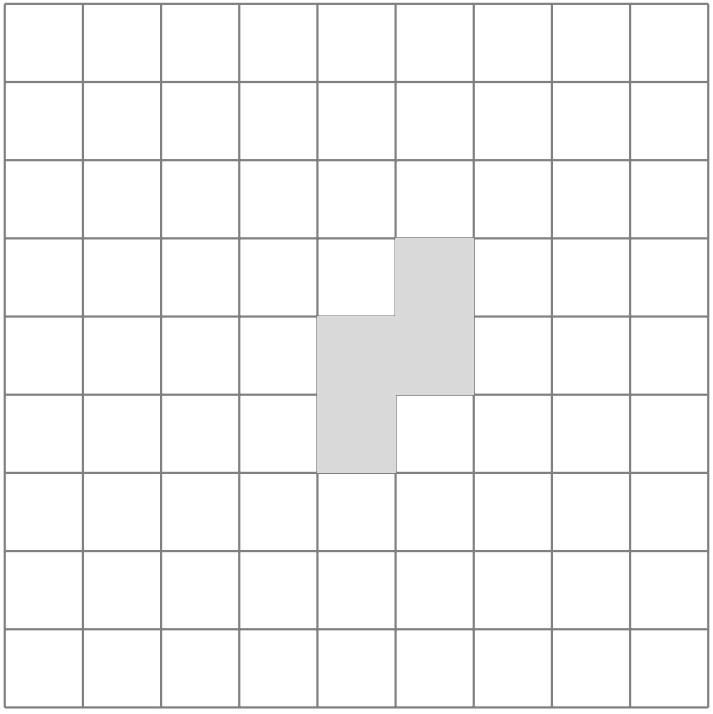
Example of pattern evolution



Example of pattern evolution

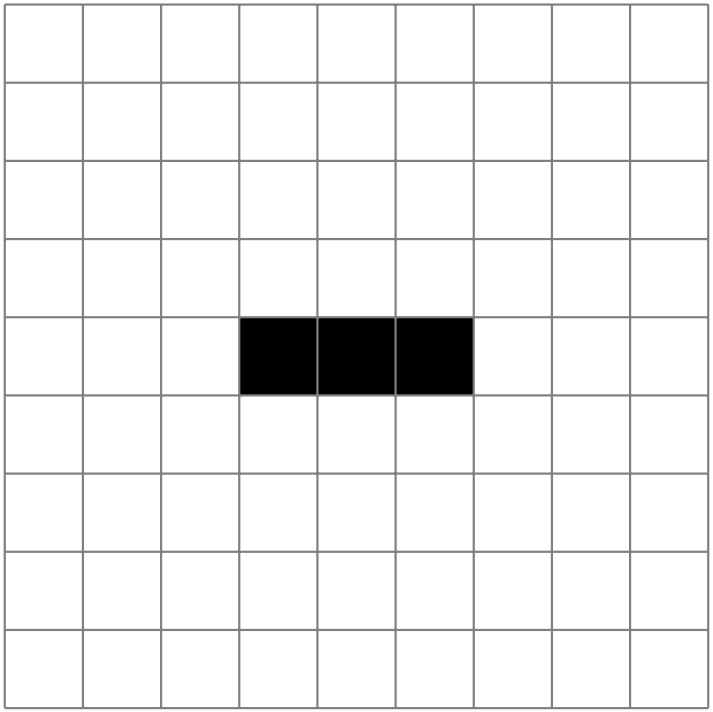


Example of pattern evolution



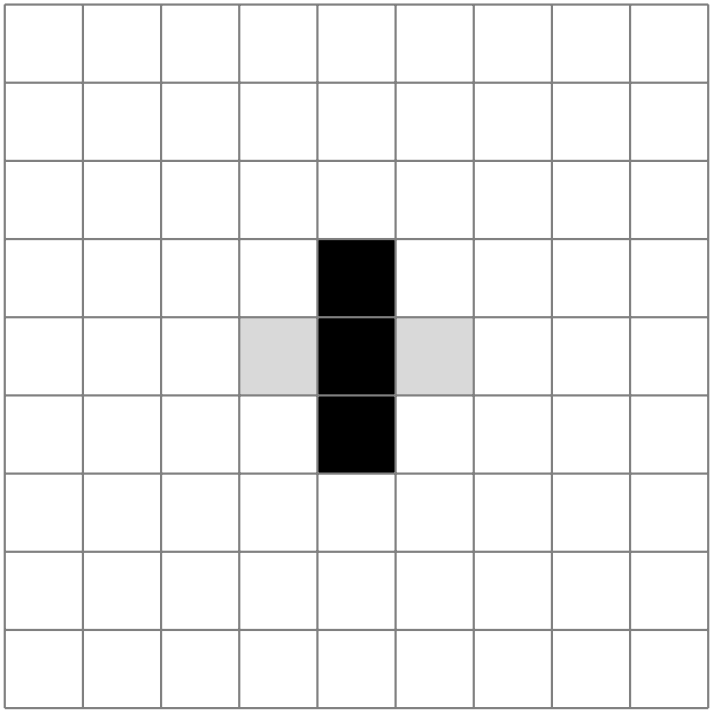
Oscillators

Periodic Life Forms or Oscillators are life forms that oscillate periodically.



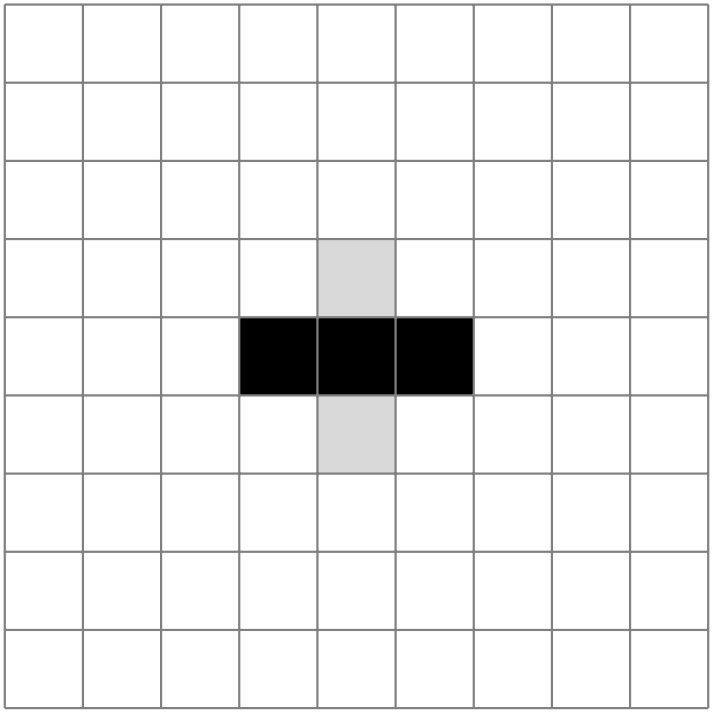
Oscillators

Periodic Life Forms or Oscillators are life forms that oscillate periodically.



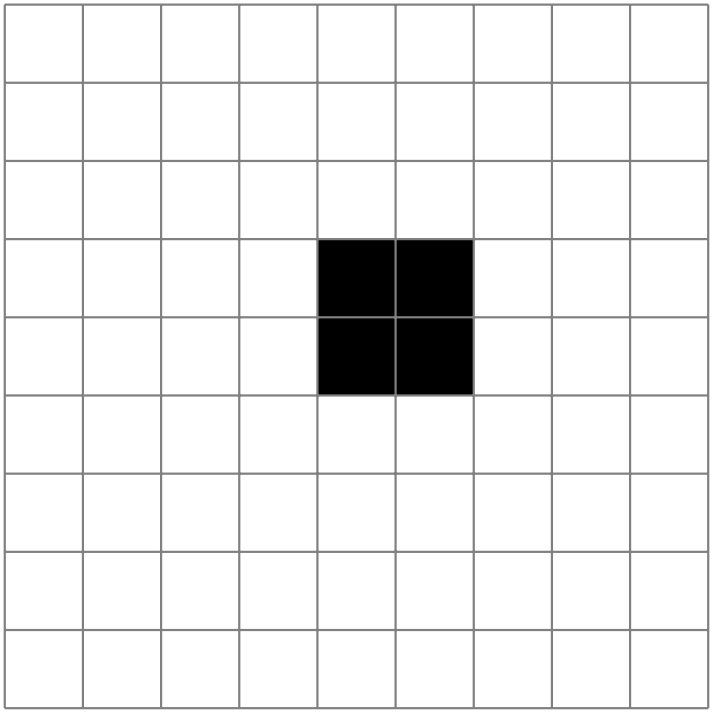
Oscillators

Periodic Life Forms or Oscillators are life forms that oscillate periodically.



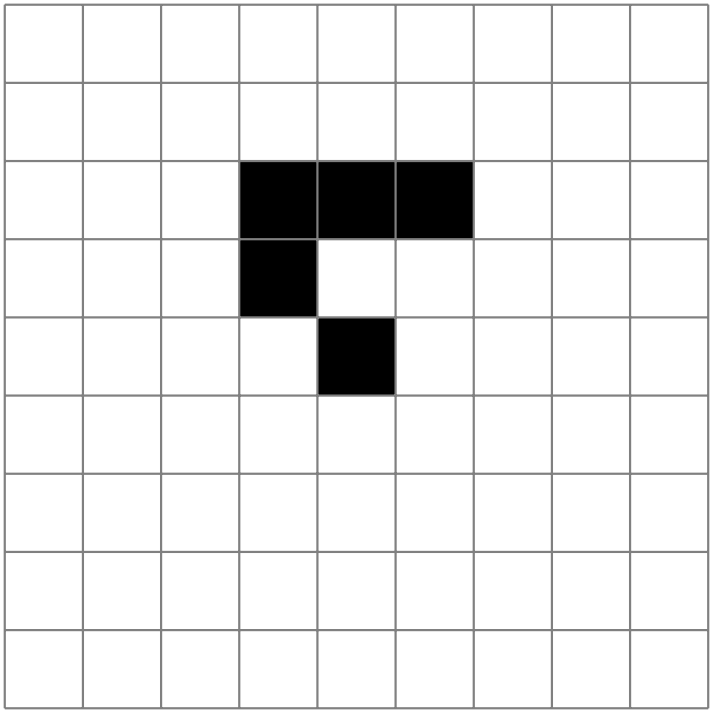
Still Life

Still Life: a stable, finite, and nonempty pattern.



Glider

A **glider** is a simple five-cell pattern that repeats itself every four generations and is offset diagonally by one cell. It is the smallest and most common type of spaceship. A spaceship is a pattern that moves across the game board.



Some Patterns in Conway's Game of Life

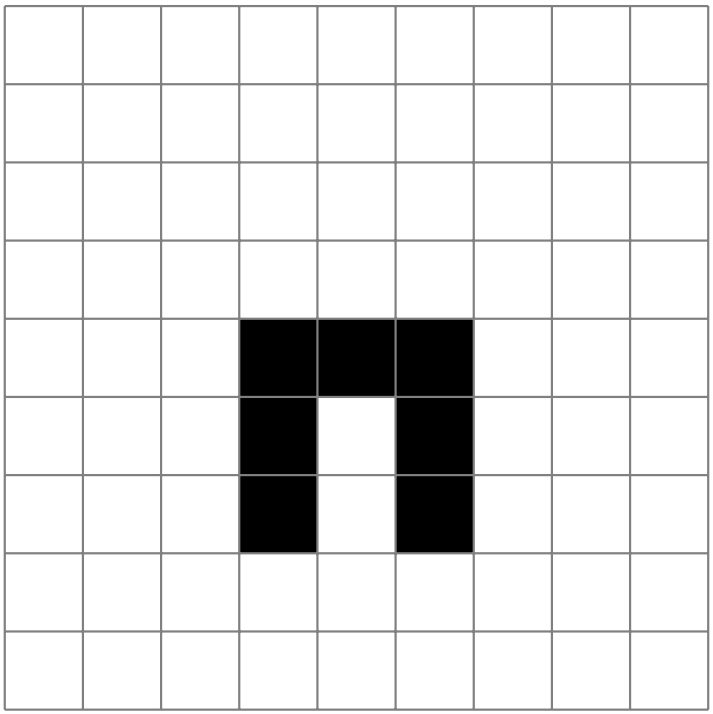
- **Oscillators:** Patterns that return to their initial state after a certain number of generations.
- **Spaceships:** Patterns that translate themselves across the board.
- **Gardens of Eden:** Configurations that cannot be reached from any other, meaning they have no predecessors.
- **Methuselahs:** Start with a small configuration and evolve over many generations before stabilizing.
- **Eaters:** Stationary or oscillating patterns that can destroy other patterns. Used to 'eat' gliders and other spaceships.
- **Rakes:** Patterns that move like spaceships but also emit other patterns, typically gliders.
- **Guns:** Stationary patterns that periodically emit spaceships or other patterns.
- **Memory Cell:** Stores information that can be read out.

Constructing Logical Gates in Conway's Game of Life

- **Computational Universality:** Conway's Game of Life is Turing complete, meaning it can simulate a universal constructor or any Turing machine.
- **Logic Gates:** Using gliders and eaters, specific logic gates like AND, OR, NOT can be constructed.
  - **AND Gate:** Two gliders collide in a way that if both are present, a new glider is formed (output).
  - **OR Gate:** Gliders are sent such that if at least one arrives, a new glider is formed.
  - **NOT Gate:** Utilizes a glider stream that is blocked unless another glider interferes, allowing a glider to pass through if no input glider is present.
- **Application:** These gates can be used to build more complex circuits and perform computations.
- A computer with memory, a processor, and a display can be built in Conway's Game of Life.

Task: How Will This Pattern Evolve?

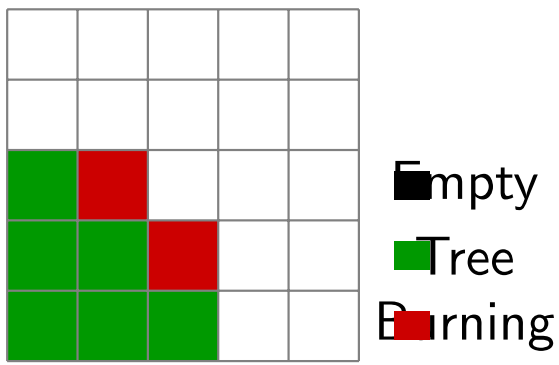
- It is **impossible** to predict, in advance, what behavior will be displayed by the Cellular Automata given a set of rules.
- We will find out how the following shape evolves in the exercise.



Forest Fire Model

The Forest Fire Cellular Automaton

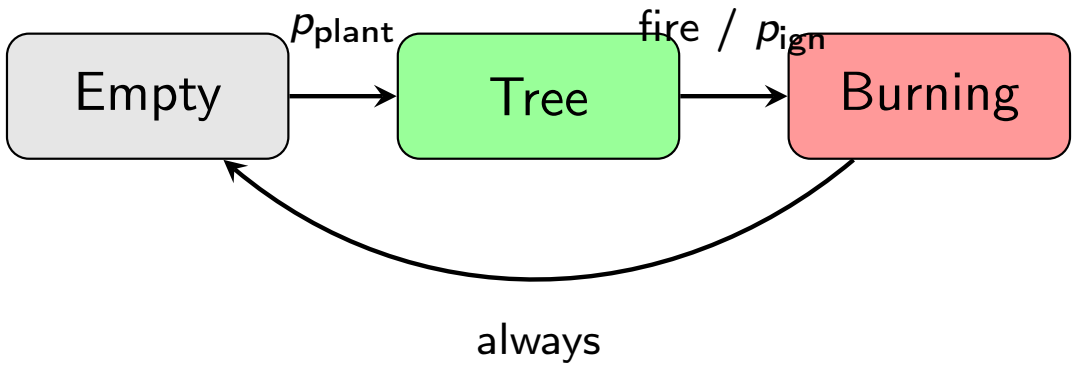
- A classic CA model that simulates the spread of fire through a forest.
- Each cell is in one of three states:
  - **Empty** (0) — bare ground
  - **Tree** (1) — living tree
  - **Burning** (2) — tree on fire
- The model captures the interplay between **growth**, **ignition**, and **fire spread**.



Rules of the Forest Fire Model

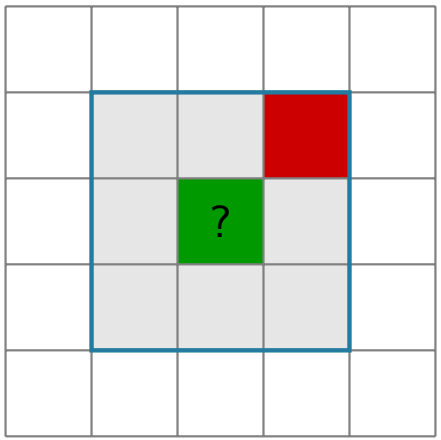
At each time step, **every cell is updated simultaneously**:

1. **Burning** $\rightarrow$ **Empty**: A burning tree burns out and becomes empty ground.
2. **Tree** $\rightarrow$ **Burning**: A tree catches fire if **any neighbor is burning**.
3. **Tree** $\rightarrow$ **Burning**: A tree may also ignite **spontaneously** with probability p_{ignition} (lightning strike).
4. **Empty** $\rightarrow$ **Tree**: An empty cell grows a new tree with probability p_{planting}.



Detecting Fire in the Neighbourhood

- We use the **Moore neighbourhood** (8 surrounding cells).
- A helper function checks whether **any** neighbour has state 2 (burning).



The tree at the centre has a burning neighbour $\Rightarrow$ it will catch fire in the next step.

Updating a Single Cell

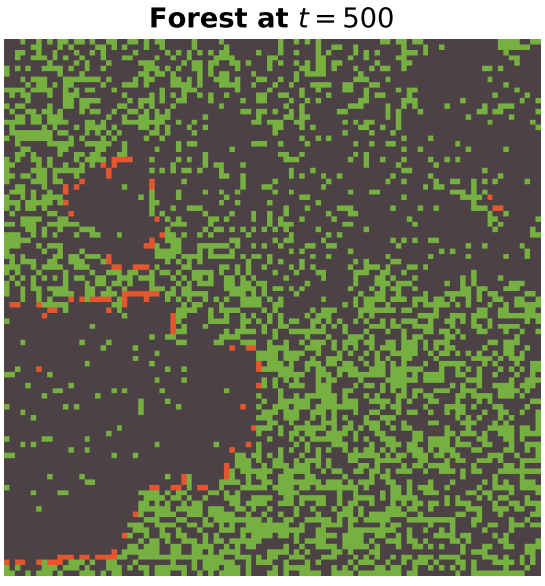
State 0 (Empty) $\rightarrow$ Tree with prob p_{plant}

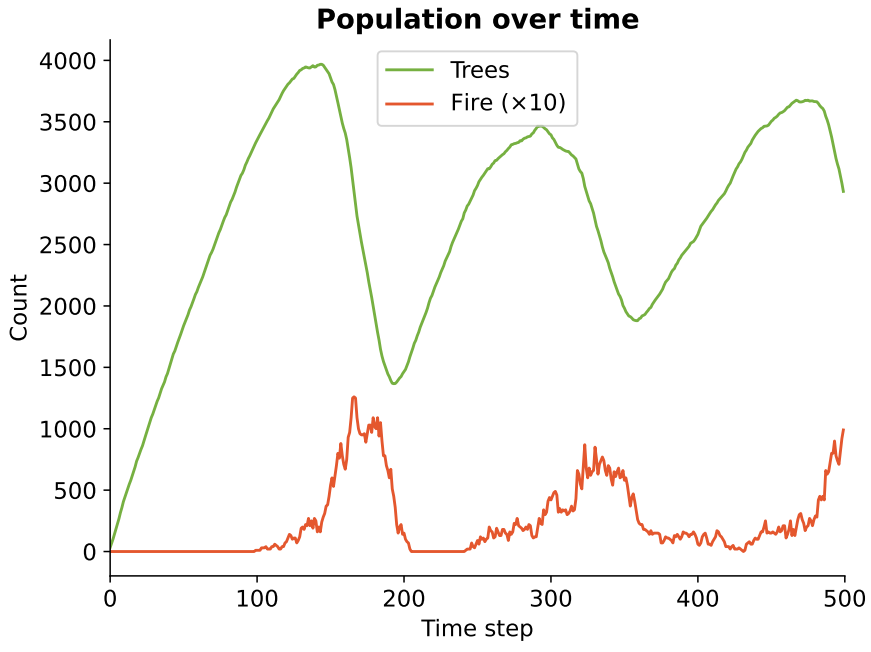
State 1 (Tree) $\rightarrow$ Burning if fire nearby or lightning

State 2 (Burning) $\rightarrow$ Empty (always)

```
def update_cell_state(state, fire_nearby,
                      p_plant, p_ignition):
    if state == 0:
        return 1 if random() < p_plant else 0
    if state == 1:
        if fire_nearby or random() < p_ignition:
            return 2
        return 1
    return 0  # burning -> empty
```

Simulation Results





Left: spatial grid at a snapshot. Right: tree count and fire count over time. The system exhibits **self-organised criticality** — large fires occur unpredictably.

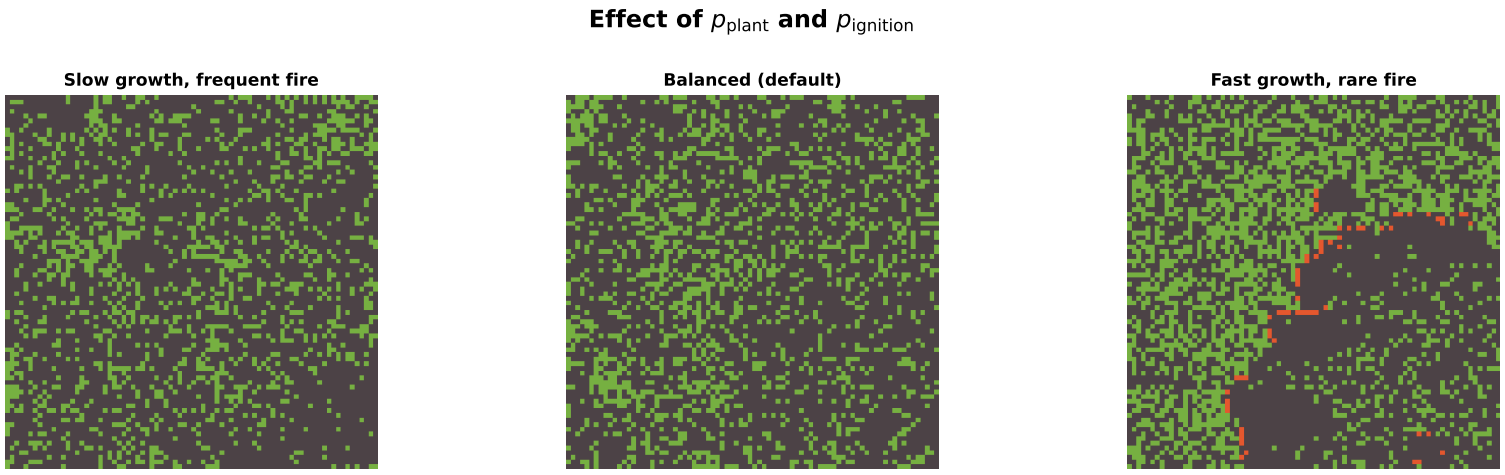
Self-Organised Criticality

- Trees gradually fill the grid until a **critical density** is reached.
- A single lightning strike can then trigger a **large-scale fire**.
- After the fire, density drops and the cycle repeats.

Self-Organised Criticality

- Trees gradually fill the grid until a **critical density** is reached.
- A single lightning strike can then trigger a **large-scale fire**.
- After the fire, density drops and the cycle repeats.
- This is an example of **self-organised criticality** (SOC):
  - The system drives itself to a critical state without external tuning.
  - Similar patterns appear in earthquakes, avalanches, and stock markets.
- The distribution of fire sizes follows a **power law**.

Effect of Parameters



Varying the ratio $p_{plant}/p_{ignition}$ changes the frequency and size of fires.

Exercise: Forest Fire

1. Implement the `detect_fire` function that checks for burning neighbours.
2. Implement the `update_cell_state` function.
3. Run the simulation and observe the dynamics.
4. Experiment: what happens when you change p_{plant} and p_{ignition}?
5. Advanced: add **wind direction** — fire spreads more easily downwind.

Elementary Cellular Automata

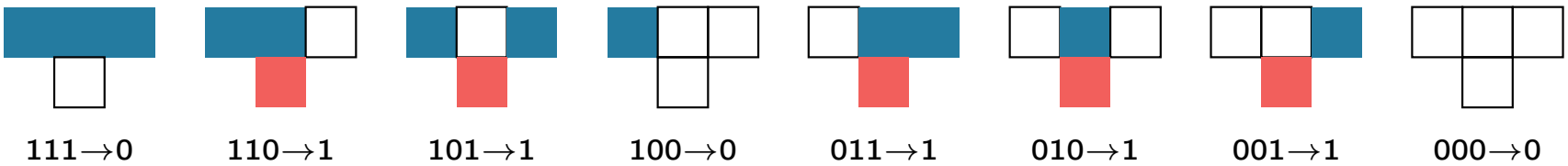
What Are Elementary Cellular Automata?

- The simplest possible class of cellular automata.
- **1D grid** — a row of cells, each either 0 (white) or 1 (black).
- Each cell's next state depends on its **current state** and its **two immediate neighbours**.
- There are $2^3 = 8$ possible neighbourhood patterns, each mapped to 0 or 1.
- Therefore there are $2^8 = 256$ possible rules, numbered 0–255.

neighbourhood

The Wolfram Numbering System

Each rule is defined by its output for all 8 possible 3-cell patterns:

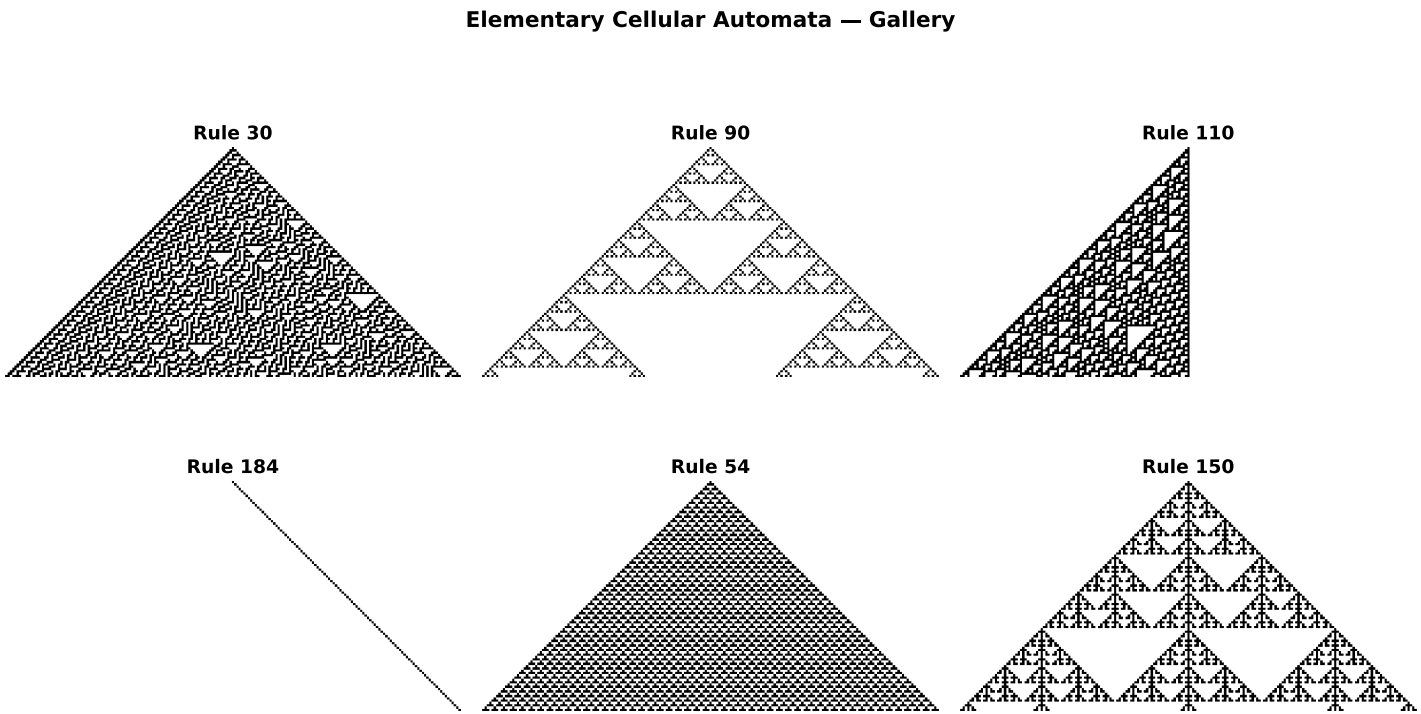


Example: **Rule 110** has outputs $0, 1, 1, 0, 1, 1, 1, 0 \Rightarrow$ binary 01101110 $= 110$.

- Stephen Wolfram systematically studied all 256 rules.
- Despite their simplicity, some rules produce remarkably complex behaviour.

Visualising Elementary CA

- Start with a single black cell in the centre of a row.
- Apply the rule to produce the next row.
- Stack rows top to bottom — time flows downward.
- The resulting 2D image reveals the rule's behaviour.



Wolfram's Four Classes

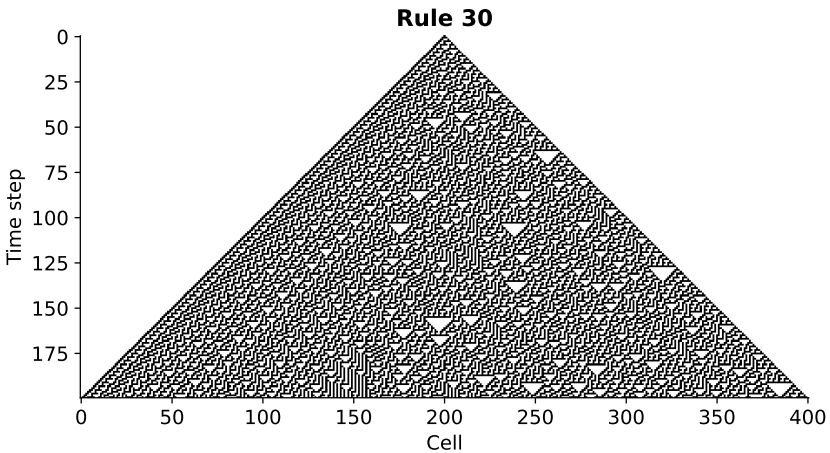
1. **Class I** — Evolution leads to a **uniform** state (e.g. Rule 0, Rule 32).
2. **Class II** — Evolution leads to **periodic** or **stable** structures (e.g. Rule 4, Rule 108).
3. **Class III** — Evolution leads to **chaotic**, aperiodic behaviour (e.g. Rule 30, Rule 45).
4. **Class IV** — Evolution leads to **complex localised structures** (e.g. Rule 110, Rule 54).

Wolfram's Four Classes

1. **Class I** — Evolution leads to a **uniform** state (e.g. Rule 0, Rule 32).
2. **Class II** — Evolution leads to **periodic** or **stable** structures (e.g. Rule 4, Rule 108).
3. **Class III** — Evolution leads to **chaotic**, aperiodic behaviour (e.g. Rule 30, Rule 45).
4. **Class IV** — Evolution leads to **complex localised structures** (e.g. Rule 110, Rule 54).

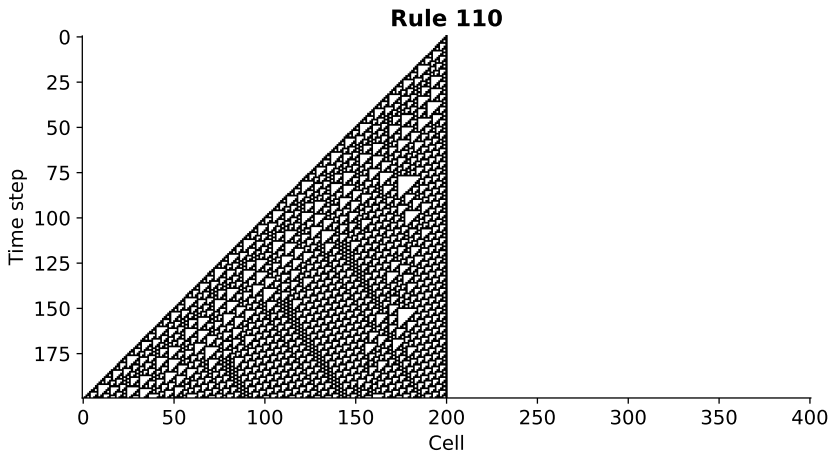
- Class IV is the most interesting — it sits at the **edge of chaos**.
- **Rule 110** has been proven to be **Turing complete** — it can perform any computation.

Rule 30 — Chaos from Simplicity



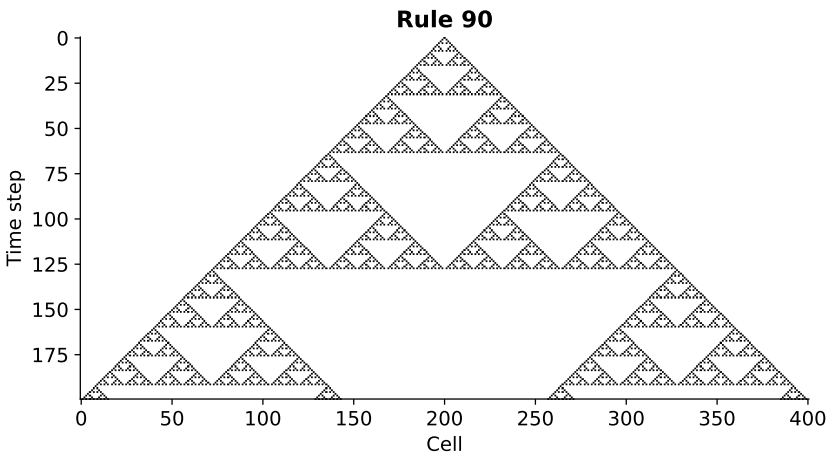
- Rule 30 produces **chaotic** patterns from a single black cell.
- The central column passes statistical tests for **randomness**.
- Wolfram used it as a random number generator in *Mathematica*.
- The pattern appears on the shell of the *Conus textile* sea snail.

Rule 110 — Complexity and Computation



- Rule 110 produces a mixture of **regular** and **complex** structures.
- Gliders, oscillators, and other structures interact.
- Proven **Turing complete** by Matthew Cook in 2004.
- This means a 1D, 2-state, 3-neighbour CA can simulate **any computer**.

Rule 90 — Sierpinski Triangle



- Rule 90 produces a perfect **Sierpinski triangle**.
- This is a well-known **fractal**.
- The rule is equivalent to **XOR** of the two neighbours.
- Connection to Lesson 5: fractals and chaos from simple rules.

Implementing an Elementary CA

```python
def rule_to_table(rule_number):
    """Convert rule number to lookup table."""
    return {(i>>2 & 1, i>>1 & 1, i & 1):
            (rule_number >> i) & 1
            for i in range(8)}

def step_eca(row, table):
    """Apply one step of the ECA rule."""
    n = len(row)
    new = np.zeros(n, dtype=int)
    for i in range(n):
        left   = row[(i - 1) % n]
        center = row[i]
        right = row[(i + 1) % n]
        new[i] = table[(left, center, right)]
    return new
```

Exercise: Elementary Cellular Automata

1. Implement the rule lookup table and the step function.
2. Generate the space-time diagram for Rule 30, Rule 90, and Rule 110.
3. Classify each rule according to Wolfram's four classes.
4. Explore: which other rules produce interesting patterns? Try Rules 54, 60, 150, 184.
5. Advanced: implement Rule 184 as a **traffic model** — interpret 1 as a car and observe traffic jams.